

Práctica

Programación Orientada a Objetos (POO)

Clases y Objetos

Problema 1:

Desarrollar un programa que contenga una clase **Cilindro** y contenga los siguientes atributos y métodos:

Cilindro	Clase
Pi	Atributo(s) de Clase
Radio Altura	Atributo(s) de instancia
Superficie(). $S=2\pi rh+2\pi r^2$ Volumen(). $V=\pi r^2 h$	Métodos

Ejemplo:

Entrada: r=5, h=20 salida: S=785.4 V= 1570.79

Problema 2:

Dados el nombre y el año de nacimiento de una persona, implementar la clase **Persona** y crear el objeto que muestre en pantalla la edad actual de esa persona.

Los atributos y métodos de la clase son:

Persona	Clase
Nombre Año_de_nacimiento	Atributo(s) de instancia
Edad_Actual()	Métodos

Nota1: Las instrucciones:

```
import datetime
datetime.date.today().year
```

devuelven el año de la fecha del sistema

Ejemplo1:

Fecha_nacimiento = 20/12/2000
Fecha_actual = 28/04/2022
Edad = 21 años

Ejemplo2:

Fecha_nacimiento = 20/04/2000
Fecha_actual = 28/04/2022
Edad = 22 años

Problema 3:

La unidad de patrimonio histórico (PRAHS) requiere registrar los siguientes datos: cantidad de cuartos, puertas y ventanas de aquellos inmuebles de color blanco, además de su superficie en m^2 . Para ello se solicita crear un programa que contenga la definición de la clase **MiCasa** y la creación de un objeto que calcule:

- El precio total de la vivienda a un precio comercial de 450 \$u\$ el m^2 construido y 280 \$u\$ el m^2 sin construir.
- El tamaño final de la superficie del inmueble si se realiza una expansión del 50%.
- El tamaño final de la superficie del inmueble si se realiza una reducción del 35%.

La clase, los atributos y métodos son:

MiCasa	Clase
Color	Atributo(s) de clase
Cuartos Puertas Ventanas Superficie	Atributo(s) de instancia
Precio() Expansión() Reducción()	Métodos

Ejemplo: Construido_m2 = 150, Sin_Construir_m2=150

Precio total = $67500 + 42000 = 109500$

Tamaño final(expansión 50%) = $150 \cdot 1.5 = 225$

Tamaño final (reducción 35%) = $225 \cdot 0.65 = 146.25$

Problema 4:

Un banco tiene 3 clientes que pueden hacer depósitos y retiros. Simular estas operaciones. Así mismo el banco requiere que al final del día se calcule el saldo final de cada cliente.

Los atributos y los métodos de cada clase son:

Cliente	Clase
Nombre Monto	Atributo(s) de instancia
Deposito (Monto) Retiro (Monto) ImprimirSaldo()	Métodos

Ejemplo:

Saldo inicial = 100

Saldo Final al 2022-04-27 = 600.00 Bs.

Nombre: Mario Saavedra

Saldo inicial = 200.5

Saldo Final al 2022-04-27 = 100.50 Bs.

Nombre: Jaime Ovando

Saldo inicial = 1600

Saldo Final al 2022-04-27 = 0.00 Bs.

Problema 5:

Juego de Cartas: Un jugador recibe 5 cartas, determinar:

- Cuantos pares sacó (par: dos cartas iguales)
- Si su juego contiene una terna (terna: tres cartas iguales)
- Si obtuvo poker (poker: cuatro cartas iguales)

Definir la clase, la creación de objeto(s) e implementar los atributos y métodos necesarios.

Problema 6:

Una empresa de construcción sostenible quiere probar nuevas mezclas de concreto ecológico para sus aceras. Cada mezcla está formada por:

- Cemento (kg)
- Arena (kg)
- Grava (kg)
- Vidrio reciclado (kg)

Además, para cada material se tiene un precio por kilogramo.

La empresa requiere:

- Definir una clase **ConcreteMix** que tenga atributos para el peso y precio de cada material (8 atributos en total).
- Un método **peso_total()** que sume los cuatro materiales.
- Un método **costo_total(tasa_transporte=0.0)** que permita añadir un porcentaje adicional **opcional** al costo total.
- Un método **porcentaje_reciclado()** que calcule el porcentaje de vidrio reciclado.
- Un método llamado **combinar_mixes(mix2)** que reciba **otra mezcla** como parámetro, sume las cantidades de los materiales y retorne una **nueva instancia combinada** (asumiendo que ambos tienen los mismos precios).
- En el programa principal:
 - Crea dos objetos **Mezcla1** y **Mezcla2** pasando a través del constructor los 8 valores por cada uno.
 - Muestra los datos de la **Mezcla1** (**peso_total**, **costo_total** sin transporte, **costo_total** con transporte con una tasa de 10%, **porcentaje reciclado**

- c. Muestra los datos de la **instancia combinada** (peso_total, costo_total con transporte, porcentaje reciclado)

Requerimientos adicionales:

- No usar colecciones como listas, diccionarios o tuplas.
- No usar generadores (i for i in range(1, 6)) , ni emplear el símbolo *args.

Problema 7:

Imagina que eres el encargado de diseñar un sistema para una innovadora cafetería llamada "El Rincón del Gamer". Esta cafetería no solo sirve deliciosas bebidas y aperitivos, sino que también ofrece a sus clientes la posibilidad de alquilar consolas de videojuegos por horas. Necesitas diseñar un sistema que permita gestionar tanto los productos que ofrece la cafetería como las consolas disponibles para alquiler, así como realizar el seguimiento de los alquileres activos. Para coadyuvar con tu tarea te propongo el código que implementa las clases.

#Código - Python

class Producto:

```
def __init__(self, nombre, precio, categoria):
    self.nombre = nombre
    self.precio = precio
    self.categoria = categoria
```

```
def mostrar_info(self):
    print(f"Nombre: {self.nombre}")
    print(f"Precio: ${self.precio:.2f}")
    print(f"Categoría: {self.categoria}")
```

class Alquiler:

```
def __init__(self, consola, nombre_cliente, hora_inicio, duracion):
    self.consola = consola
    self.nombre_cliente = nombre_cliente
    self.hora_inicio = hora_inicio
    self.duracion = duracion
```

```
def calcular_costo(self):
    return self.consola.precio * self.duracion
```

```
def mostrar_detalle(self):
    print("\n--- Detalles del Alquiler ---")
    print(f"Cliente: {self.nombre_cliente}")
    print("Consola:")
    self.consola.mostrar_info()
    print(f"Hora de Inicio: {self.hora_inicio}")
```

```
print(f"Duración: {self.duracion} horas")
print(f"Costo Total: ${self.calcular_costo():.2f}")

# Ejemplo de uso
cafe = Producto("Café Americano", 2.50, "Bebida")
tostada = Producto("Tostada Integral", 3.00, "Aperitivo")
xbox = Producto("Xbox Series X", 5.00, "Consola")
switch = Producto("Nintendo Switch", 4.00, "Consola")

cafe.mostrar_info()
print("\n")
xbox.mostrar_info()

alquiler1 = Alquiler(xbox, "Ana", "16:00", 3)
alquiler1.mostrar_detalle()

alquiler2 = Alquiler(switch, "Carlos", "17:30", 2)
alquiler2.mostrar_detalle()
```

Completa el programa agregando sobre el bloque principal:

a. La creación de los objetos:

Objeto	Nombre	precio	categoria
cafe	Café Americano	2.50	Bebida
tostada	Tostada Integral	3.00	Aperitivo
xbox	Xbox Series X	5.00	Consola
switch	Nintendo Switch	4.00	Consola

- b. Muestra la información de los objetos **café** y **xbox**
- c. Crea el objeto **alquiler1**, el producto alquilado es un **xbox**, lo alquilo **Ana** a hrs. **16:00** durante **3** horas. Muestra los detalles del alquiler1.
- d. Crea el objeto **alquiler2**, el producto alquilado es un **switch**, lo alquilo **Carlos** a hrs. **17:30** durante **2** horas. Muestra los detalles del alquiler2.

Problema 8:

En una prueba técnica para optar al cargo de Desarrollador Junior en Google, se proponen evaluar tu creatividad enunciando este desafío de programación. En la villa ficticia de **Villa Imaginaria** habitan diferentes criaturas fantásticas, cada una con características y habilidades propias. Queremos que construyas un programa orientado a objetos que permita crear estas criaturas, mostrar sus habilidades especiales y simular interacciones entre ellas.

Requerimientos:

1. **Definir clases independientes para cada tipo de criatura**
 - Crea al menos dos clases distintas, por ejemplo, *Hipogrifo* y *Unicornio*.
 - Cada clase debe tener atributos propios, incluyendo:
 - nombre (cadena)
 - edad (entero)
 - Un atributo específico que distinga a esa criatura (por ejemplo, `color_fuego` para Hipogrifo y `poder_magico` para Unicornio).
 - Un atributo tipo que identifique el tipo de criatura (por ejemplo, "Hipogrifo" o "Unicornio").
2. **Implementar un método para mostrar la acción especial de cada criatura**
 - Este método debe imprimir un mensaje que describa la habilidad única de la criatura.
3. **Implementar un método en cada clase para la interacción entre dos criaturas**
 - El método debe recibir como parámetro otra criatura.
 - Según el valor del atributo tipo de la criatura con la que se interactúa, se debe imprimir un mensaje específico que refleje la interacción entre ambas criaturas.
 - Si la criatura no reconoce el tipo de la otra, debe imprimir un mensaje indicando que no sabe cómo interactuar.
4. **Programa principal**
 - Crea al menos dos objetos, uno de cada clase (usa el constructor de la clase).
 - Muestra la acción especial de cada uno.
 - Simula la interacción entre ellos en ambas direcciones (cada criatura interactúa con la otra).

Restricciones

- No se debe usar herencia ni polimorfismo. Cada clase debe ser independiente.
- La interacción debe basarse en el atributo tipo para decidir qué mensaje mostrar.
- El código debe estar bien comentado y organizado.
- No se podrán emplear colecciones (listas, tuplas o diccionarios), ni generadores o expresiones regulares.